

DISK ON RAM

PLAYING WITH BLOCK DRIVERS



This article, which is part of the series on Linux device drivers, experiments with a dummy hard disk on RAM to demonstrate how block drivers work.

Part—15

After a delicious lunch, theory makes the audience sleepy. So, let's start with the code itself. The code demonstrated is available at http://linuxforu.com/article_source_code/feb12/device_driver.zip. This tar ball contains three 'C' source files, two 'C' headers, and *Makefile*. As usual, executing *make* will build the 'disk on RAM' driver (*dor.ko*), combining the three 'C' files. Check out the *Makefile* to see how. To clean the built files, run the usual *make clean*.

Once built, the following are the experimental steps (refer to Figures 1 to 3). Please note that all these need to be executed with root privileges:

- Load the driver *dor.ko* using *insmod*. This would create the block device files representing the disk on 512 KiB of RAM, with three primary and three logical partitions.
- Check out the automatically created block device files (*/dev/rb**). */dev/rb* is the entire disk, which is 512 KiB in size. *rb1*, *rb2* and *rb3* are the primary partitions, with *rb2* being the extended partition and containing three logical partitions *rb5*, *rb6* and *rb7*.
- Read the entire disk (*/dev/rb*) using the disk dump utility *dd*.
- Zero out the first sector of the disk's first partition (*/dev/rb1*), again using *dd*.
- Write some text into the disk's first partition (*/dev/rb1*) using *cat*.
- Display the initial contents of the first partition (*/dev/rb1*) using the *xxd* utility. See Figure 2 for *xxd* output.
- Display the partition information for the disk using *fdisk*. See Figure 3 for *fdisk* output.
- Quick-format the third primary partition (*/dev/rb3*) as a *vfat* filesystem (like your pen drive), using *mkfs.vfat* (Figure 3).
- Mount the newly formatted partition using *mount*, say at */mnt* (Figure 3).
- The disk usage utility *df* would now show this partition mounted at */mnt* (Figure 3). You may go ahead and store files there, but remember that this is a disk on RAM, and so is non-persistent.

- Unload the driver using '*rmmod dor*' after unmounting the partition using '*umount /mnt*'. All data on the 'disk' will be lost.

Now let's learn the rules

We have just played around with the disk on RAM (DOR), but without actually knowing the rules, i.e., the internal details of the game. So, let's dig into the nitty-gritty to decode the rules. Each of the three .c files represent a specific part of the driver; *ram_device.c* and *ram_device.h* abstract the underlying RAM operations like *vmalloc/vfree*, *memcpy*, etc, providing disk operation APIs like *init/cleanup*, *read/write*, etc. *partition.c* and *partition.h* provide the functionality to emulate the various partition tables on the DOR. Recall the pre-lunch session (i.e., the previous article) to understand the details of partitioning. The code in this is responsible for the partition information like the number, type, size, etc, that is shown using *fdisk*. The *ram_block.c* file is the core block driver implementation, exposing the DOR as the block device files (*/dev/rb**) to user-space. In other words, four of the five files *ram_device.** and *partition.** form the horizontal layer of the device driver, and *ram_block.c* forms the vertical (block) layer of the device driver. So, let's understand that in detail.

The block driver basics

Conceptually, the block drivers are very similar to character drivers, especially with regards to the following:

- Usage of device files
- Major and minor numbers
- Device file operations
- Concept of device registration

So, if you already know character driver implementation, it would be easy to understand block drivers. However, they are definitely not identical. The key differences are as follows:

- Abstraction for block-oriented versus byte-oriented devices.
- Block drivers are designed to be used by I/O schedulers, for optimal performance. Compare that with character drivers